Feinschliff

Lernunterlage: öffentliches Repo aufräumen, einheitliche Zustände, kleine Fehler finden, README als Produktseite

1 Worum es geht

Vor der Präsentation sollte das Projekt nicht nur funktionieren, sondern **fertig wirken**: keine privaten Daten im öffentlichen Repo, jede Seite verhält sich beim Laden und bei Fehlern gleich, nichts überlappt auf dem Handy, und die README erklärt in einer Minute, was die App kann. Neue große Funktionen gab es dabei keine.

Schritt	Was	Warum
REVIEW N-1	IP, Rechner- und Tailnet-Namen entfernt	Das Repo ist öffentlich
Durchklicken	Alle Seiten im Demo, Desktop und 390 px	Fehler sieht man, nicht der Compiler
Zustände	Laden, Fehler, leer überall gleich	Wiedererkennbar, barrierearm
Setup-Endpunkt	„Erstes Konto anlegen“ nur ohne Benutzer	Kein toter Link mehr
macOS	Skripte portabel gemacht	Zweiter Rechner, gleiches Ergebnis
README	Produktseite mit Screenshots	Erster Eindruck für Lehrer und Besucher

2 Öffentliches Repo aufräumen (REVIEW N-1)

Im Repo standen eine Tailscale-IP (in `tuya-rohwerte-anzeigen.sh`) und der echte Rechnername (in der Lern-PDF zum Betrieb). Beides verrät Teile des privaten Netzes. Jetzt stehen dort Platzhalter wie `<rechner>.<tailnet>.ts.net`.

Wie sucht man so etwas?

```
git grep -nIiE '100\.[0-9]+\.[0-9]+\.[0-9]+|\.ts\.net|tailnet'   # Textdateien
python3 pdfsuche.py                                              # Text aus den PDFs (pypdf)
```

PDFs muss man eigens durchsuchen: `git grep` sieht nur den gepackten Inhalt. Die PDF wurde aus dem korrigierten Python-Skript neu erzeugt, nicht von Hand bearbeitet.

> **Historie nicht umschreiben:** In alten Commits steht die IP weiter. Entfernen ginge nur mit Force-Push, der alle Klone kaputt macht. Die Werte sind nicht geheim (kein Passwort), nur privat. Bei einem echten Geheimnis gilt dagegen: **sofort austauschen** (rotieren), das Löschen aus der Historie allein reicht nie.

Nebenbei: Token nicht in der Prozessliste

```
# vorher: Token als Argument, für andere Prozesse in "ps" sichtbar
curl -H "Authorization: Bearer $TOKEN" "$API/..."
# jetzt: Header über stdin ("-H @-")
printf 'Authorization: Bearer %s\n' "$TOKEN" | curl -sS -H @- "$API/..."
```

3 Durchklicken mit einem Skript

Ein kleines Python-Skript mit **Playwright** (steuert einen echten Browser ohne Fenster) meldet sich im Demo-Modus an, öffnet jede Seite in 1280 px und 390 px Breite, macht ein Foto und sammelt Konsolenfehler. Außerdem prüft es, ob die Seite breiter als das Fenster ist (horizontales Scrollen = Layoutfehler).

Gefunden	Ursache	Lösung
Langer Gerätename überlappt „online“	LED schwebt absolut oben rechts, Name hat keinen Platz frei	LED als eigene Zeile unter dem Gerätetyp
Rollen-Auswahl auf dem Handy nur ein Punkt	Tabellenspalte wird zusammengedrückt	min-width: 120px, die Tabelle scrollt selbst
„sonst alle 15 Minuten“	Text älter als der Takt pro Gerät	Verweis auf die Einstellungen
„Erstes Konto anlegen“ trotz Konten	Frontend wusste nicht, ob es Benutzer gibt	GET /api/auth/setup

Das Datum „11/03/2026“ im ersten Foto war **kein** Fehler der App: Der Test-Browser lief auf Englisch. Mit `locale="de-DE"` stand dort 03.11.2026. Erst prüfen, dann ändern.

4 Einheitliche Zustände

Jede Seite, die Daten lädt, kennt vier Zustände. Vorher sah jeder anders aus (mal grauer Text, mal Kasten, „Erneut versuchen“ nur auf der Übersicht). Jetzt gibt es zwei kleine Bausteine in `components/StateMessages.tsx`:

Zustand	Anzeige	Screenreader
Lädt	LoadingMessage: gestrichelter Kasten „Räume werden geladen …“	role="status" (höflich)
Fehler beim Laden	ErrorMessage mit Knopf „Erneut versuchen“	role="alert" (sofort)
Fehler beim Speichern	ErrorMessage ohne Knopf (Nochmal-Laden hilft da nicht)	role="alert"
Leer	Kasten mit Text je Seite, z. B. „Noch keine Räume“	–

```
export function ErrorMessage({ message, onRetry }: ErrorMessageProps) {
  return (
    <div className="alert alert-row" role="alert">
      <span>{message}</span>
      {onRetry && (
        <button type="button" className="button" onClick={onRetry}>Erneut versuchen</button>
      )}
    </div>
  )
}
```

Wie funktioniert „Erneut versuchen“?

Der useEffect, der lädt, hängt zusätzlich an einem Zähler `reloadKey`. Der Knopf zählt ihn hoch, React führt den Effect noch einmal aus. Kein doppelter Ladecode.

```
const [reloadKey, setReloadKey] = useState(0)
useEffect(() => { /* getRooms(search) ... */ }, [search, reloadKey])
function retry() { setLoading(true); setLoadError(null); setReloadKey((key) => key + 1) }
```

> Ladefehler und Speicherfehler getrennt halten: Nur beim Laden hilft „Erneut versuchen“.

5 Der Setup-Endpunkt

```
// GET /api/auth/setup – immer offen, verrät nur: gibt es schon einen Benutzer?
private static async Task<SetupResponse> GetSetup(UserManager<User> userManager, CancellationToken ct)
{
    var hasUsers = await userManager.Users.AnyAsync(ct);
    return new SetupResponse(NeedsFirstAccount: !hasUsers);
}
```

- **AllowAnonymous:** Die Anmeldeseite fragt, bevor jemand angemeldet ist, auch mit Leseschutz.
- **Wenig verraten:** nur ein true/false, keine Anzahl, keine E-Mail.
- **Fehler im Frontend:** Antwortet der Server nicht, gilt false. Dann fehlt nur der Link, die Anmeldung geht.
- **Tests:** leere Datenbank → true, nach dem ersten Konto → false, mit Leseschutz trotzdem 200.
- **Grenze (REVIEW N-4):** Die Registrierung selbst bleibt offen, solange es keinen Benutzer gibt. Sicherer wäre ein erster Admin nur über einen lokalen Weg. Das ist der nächste Schritt.

6 Linux und macOS

Auf dem MacBook lief `./pruefen.sh` sofort. Zwei Stellen waren aber nur für Linux geschrieben:

Stelle	Linux (GNU)	Portabel
Zufallsschlüssel in einer Zeile	base64 -w0	base64 | tr -d '\n'
Schrift für die Lern-PDFs	/usr/share/fonts/truetype/dejavu/	mehrere Ordner prüfen, macOS: ~/Library/Fonts

GNU-Werkzeuge (Linux) und BSD-Werkzeuge (macOS) heißen gleich, haben aber teils andere Optionen. Typische Fallen: `sed -i`, `readlink -f`, `date -d`.

7 README als Produktseite

Die README ist die Startseite auf GitHub. Sie beantwortet jetzt in dieser Reihenfolge: **Was ist das?** (Satz + großes Bild) → **Was kann es?** (Funktionen) → **Wie sieht es aus?** (Screenshots aus dem Demo) → **Wie probiere ich es?** (Demo in einer Minute) → Architektur, Installation (Docker, Dienst, Entwickeln), Sicherheit, Grenzen, Ausblick (Push über Pulsar, MCP).

- **Screenshots nur aus dem Demo:** keine echten Geräte, Namen oder Kamerabilder im öffentlichen Repo.
- **Grenzen offen nennen:** nur lesend, Polling, inoffizielle Blink-API. Das wirkt ehrlicher als nur Highlights.
- **Mermaid:** GitHub zeichnet Diagramme aus Text. Ändert sich die Architektur, ändert man eine Zeile.

8 Kurzantworten für Rückfragen

Frage	Antwort
Warum die IP nicht aus der Historie löschen?	Force-Push zerstört alle Klone, und die IP ist kein Geheimnis. Bei einem echten Geheimnis: austauschen.
Warum ein eigener Baustein für Fehler?	Gleiches Aussehen, gleiche Barrierefreiheit (role=alert), eine Stelle zum Ändern.
Was ist role="status" vs. role="alert"?	status liest der Screenreader vor, wenn er Zeit hat; alert unterbricht sofort.
Warum Playwright?	Man sieht die echte Seite in echten Breiten, auch Fehler, die kein Test findet.
Verrät /api/auth/setup zu viel?	Nur, ob es Benutzer gibt. Das sieht man auch daran, ob Registrieren klappt.

9 Merksätze

- Öffentliches Repo: keine IPs, Hostnamen, Tailnet-Namen; PDFs mit durchsuchen.
- Geleakte Geheimnisse rotieren, nicht nur löschen.
- Jede Seite: Laden, Fehler, leer, Daten – überall gleich anzeigen.
- Ladefehler mit „Erneut versuchen“, Speicherfehler ohne.
- Erst auf Desktop und 390 px ansehen, dann „fertig“ sagen.